

Building Decision Tree By hand using steps of Python Code

Dataset for Play Tennis

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

2.2
Second if checks if the list of attributes is empty, meaning it has reached the end of the recursion, or if the DecisionTree class has reached max_depth. If this condition is satisfied, it returns a LeafNode with a value equal to the most common value of target_attr ("Playtennis").

2.3
If both if's fail, variable A is initialized as:

`A = self.choose_best_attribute(examples, target_attr, attributes)`

2.3.1

`choose_best_attribute` starts initializing 2 vars:

`best_gain = -1`
`best_attribute = None`

After this, the function starts a for loop iterating over attributes (attribute)

For each attribute, the function initializes gain and calls `information_gain`

`gain = self.information_gain(examples, attribute, examples[target_attr])`

Begin:

1. Assigns Variables and calls ID3

`df`: The entire dataset

`target` = "Playtennis"

`Attributes` = {Outlook, Humidity, Temp, Wind}

`tree.root` = `tree.ID3(df, target, attributes)`

ID3 is defined as: `ID3(self, examples, target_attr, attributes, depth=0)`

2. ID3

`examples` = `df`
`target_attr` = `target`
`attributes` = `attributes`

2.1

First if eliminates repetitions from target_attr ("Playtennis") and check it's length and checks if length is equal to one, meaning all values of target_attr belong to a single class

(in this case "yes" or "no"). If this is satisfied, returns a LeafNode with the first value of target_attr assigned to the LeafNodes value.

`LeafNode(value = first_val_of_target_attr)`

→ Here the code gets tricky so in order to keep track, we will use the values of the variables in the first iteration:

`examples` = Still our whole df: `examples`

`attribute` = First value of attributes, in this case: "Outlook"

`examples[target_attr]` = This is basically `df["Playtennis"]` and what this does is return a column of all Playtennis values.

It would look something like this →
0 No
1 No
2 Yes
3 No
: !

Let's call it `target_col`

Now `information_gain` is defined as:

`information_gain(self, S, A, target_attr)`

`S` = `examples`
`A` = `attribute`
`target_attr` = `target_col`

Now first, it initializes 2 variables

`parent_entropy` = `self.entropy(target_col)`

`weighted_entropy` = 0

Go to next Page

Now both **entropy** and **information-gain** are based on the following formulas:

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

c different values
p_i is the portion of *S* belonging to class *i*

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Note: $\text{Entropy}(S) = 0$ means all members of *S* belong to the same class

$\text{Entropy}(S) = 1$ means that members are evenly distributed

For sake of simplicity, we will only focus on how Gain works in code, and we will assume that our function **Entropy** works.

Now, calculating **entropy(target_col)** we got that **parent-entropy = 0.94028....**

Now, **information-gain** starts a for loop:

for value in **examples["Outlook"].unique()**

This basically says: "For each value in the possible values of "Outlook"

This being: ['Sunny', 'Overcast', 'Rainy']

In the first iter, **value = 'Sunny'**

Inside the loop, it initializes:

subset = examples[examples["Outlook"] == "Sunny"]

This is creating a subset of our data set where all values of "Outlook" are equal to "Sunny", giving us the following:

	Day	Outlook	Temperature	Humidity	Wind	PlayTennis
0	D1	Sunny	Hot	High	Weak	No
1	D2	Sunny	Hot	High	Strong	No
7	D8	Sunny	Mild	High	Weak	No
8	D9	Sunny	Cool	Normal	Weak	Yes
10	D11	Sunny	Mild	Normal	Strong	Yes

Next it initializes **weight** as:

$$\text{weight} = \ln(\text{subset}) / \ln(\text{examples}) = 0.35714...$$

This being $\frac{\ln 1}{\ln 5}$ from the gain function.

After this, our function sums our previous **weighted-entropy** with the following operation:

$$\text{weight} \cdot \text{Self.entropy}(\text{target_col}[\text{examples["Outlook"] == "Sunny"}])$$

Remember that **target_col** is all the values of PlayTennis, so this code says:
: Give me the entropy of "PlayTennis" when "Outlook" is "Sunny"

Now our **weighted-entropy** changed to **0.346768....**

Now, we are still in the for loop at the moment. Since we still have not iterated over "Overcast" and "Rainy". Let's skip over those steps and get our final **weighted-entropy = 0.69353....**

At the end of **information-gain** it initializes **gain = parent-entropy - weighted-entropy**

And returns **gain**. This is basically how valuable the attribute that we analyzed is.

Now we are still in the initial for loop inside of **choose_best_attribute**, so after obtaining **gain**, the function checks if **gain > best-gain**.

In this case, **gain = 0.2467....** and **best-gain = -1**

So, the if statement succeeds and reassigns **best-gain** to the value of **gain**. It also reassigns **best_attribute**

to the attribute that was analyzed, in this case it was "Outlook"

Now this for loop will run for the remaining attributes: {Temp, Humidity, Wind}

Since we've already done the steps for "Outlook", we will skip over to the end results:

best-gain = -1 / best_attribute = None

Step1: Outlook gain > best-gain so:

best-gain = 0.2467 / best_attribute = Outlook

Step2: Temp gain not > than best-gain so:

Next step

Step3: Humidity gain not > than best-gain so:

Next Step

Step4: Wind gain not > than best-gain so:

Next Step

At this point, there are no more attributes so the for loop ends and **choose_best_attribute** returns the current **best_attribute**, this being "Outlook"

So now, we have gotten our root attribute.

We now finally, initialize our first and top `InternalNode` as: `decision_node = InternalNode("Outlook", {})`.

We now start a previously seen loop:

for value in `examples["Outlook"].unique()`

This basically says: "For each value in the possible values of "Outlook"

This being: ['Sunny', 'Overcast', 'Rain']

Inside the loop we initialize `examples_v` in a previously seen way as well:

`examples[examples["Outlook"] == "Sunny"]`

Giving us:

	Day	Outlook	Temperature	Humidity	Wind	PlayTennis
0	D1	Sunny	Hot	High	Weak	No
1	D2	Sunny	Hot	High	Strong	No
7	D8	Sunny	Mild	High	Weak	No
8	D9	Sunny	Cool	Normal	Weak	Yes
10	D11	Sunny	Mild	Normal	Strong	Yes

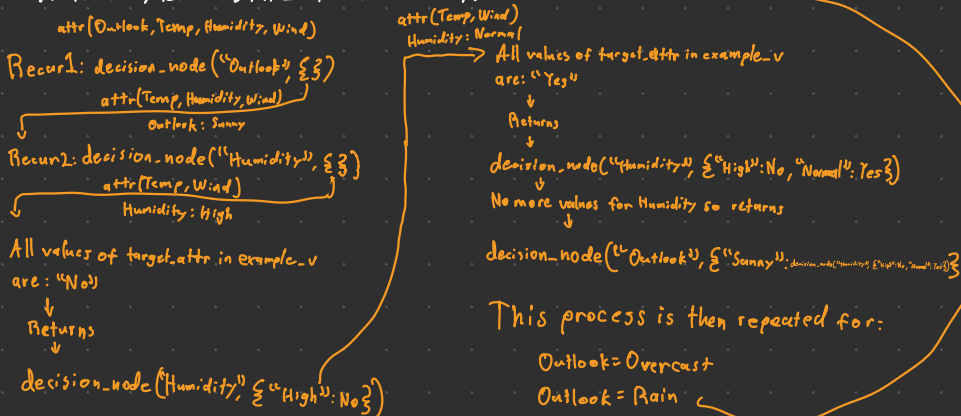
and we initialize the value of `branches` inside our `decision_node` as the result of a recursive call of the function `ID3`:

`decision_node.branches["Sunny"] = self.ID3(examples_v, target_attr, attributes - {"Outlook"}, depth+1)`

This is how we get the next `decision_node` for each outcome of our previous `decision_node`. This is basically how it draws the paths of each outcome.

When recursion ends, the function finally returns our `decision_node`.

As a final note, let's visualize the recursion:



Final result in code:

`decision_node("Outlook", {"Sunny": decision_node("Humidity", {"High": No, "Normal": Yes}), "Overcast": "Yes", "Rain": decision_node("Wind", {"Strong": No, "Weak": Yes})})`

This process is then repeated for:

`Outlook = Overcast`

`Outlook = Rain`